

Lab 4: Word Embeddings with Word2Vec

1. Thiết lập môi trường

```
!pip install gensim pyspark tqdm requests
```

Collecting gensim

Downloading gensim-4.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.1 kB)

Requirement already satisfied: pyspark in

/usr/local/lib/python3.12/dist-packages (3.5.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.1)

Requirement already satisfied: requests in

/usr/local/lib/python3.12/dist-packages (2.32.4)

Collecting numpy<2.0,>=1.18.5 (from gensim)

Downloading numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
61.0/61.0 kB 3.0 MB/s eta

0:00:00

gensim)

Downloading scipy-1.13.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
60.6/60.6 kB 4.2 MB/s eta

0:00:00

ent already satisfied: smart-open>=1.8.1 in

/usr/local/lib/python3.12/dist-packages (from gensim) (7.3.1)

Requirement already satisfied: py4j==0.10.9.7 in

/usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.7)

Requirement already satisfied: charset_normalizer<4,>=2 in

/usr/local/lib/python3.12/dist-packages (from requests) (3.4.3)

Requirement already satisfied: idna<4,>=2.5 in

/usr/local/lib/python3.12/dist-packages (from requests) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in

/usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in

/usr/local/lib/python3.12/dist-packages (from requests) (2025.10.5)

Requirement already satisfied: wrapt in

/usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.17.3)

Downloading gensim-4.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (26.6 MB)
26.6/26.6 MB 104.4 MB/s eta

0:00:00

py-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.0 MB)

18.0/18.0 MB 126.6 MB/s eta

```

0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.2 MB)
----- 38.2/38.2 MB 16.6 MB/s eta
0:00:00
py, scipy, gensim
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2
    Uninstalling numpy-2.0.2:
      Successfully uninstalled numpy-2.0.2
  Attempting uninstall: scipy
    Found existing installation: scipy 1.16.2
    Uninstalling scipy-1.16.2:
      Successfully uninstalled scipy-1.16.2
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
opencv-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >=
"3.9", but you have numpy 1.26.4 which is incompatible.
opencv-contrib-python 4.12.0.88 requires numpy<2.3.0,>=2;
python_version >= "3.9", but you have numpy 1.26.4 which is
incompatible.
opencv-python-headless 4.12.0.88 requires numpy<2.3.0,>=2;
python_version >= "3.9", but you have numpy 1.26.4 which is
incompatible.
tsfresh 0.21.1 requires scipy>=1.14.0; python_version >= "3.10", but
you have scipy 1.13.1 which is incompatible.
thinc 8.3.6 requires numpy<3.0.0,>=2.0.0, but you have numpy 1.26.4
which is incompatible.
Successfully installed gensim-4.3.3 numpy-1.26.4 scipy-1.13.1

{"id": "edfc6907b94543c6bd19a6ff4552e6c8", "pip_warning": {"packages":
["numpy"]}}

import re
from pathlib import Path
from typing import List, Optional

import numpy as np
from gensim import downloader as api

class WordEmbedder:
    def __init__(self, model_name: str = "glove-wiki-gigaword-50") ->
None:
        self.model_name = model_name
        self.model = api.load(model_name)
        self.vector_size = self.model.vector_size
        print(f"Đã tải mô hình '{model_name}' với vector_size =
{self.vector_size}")

    @staticmethod

```

```

def _basic_tokenize(text: str) -> List[str]:
    tokens = re.findall(r'[\w\']+', text.lower())
    return tokens

def get_vector(self, word: str) -> Optional[np.ndarray]:
    if word in self.model:
        return np.array(self.model[word])
    return None

def get_similarity(self, word1: str, word2: str) ->
Optional[float]:
    """Tính cosine similarity giữa hai từ, bỏ qua nếu 00V."""
    if word1 not in self.model or word2 not in self.model:
        return None
    return float(self.model.similarity(word1, word2))

def get_most_similar(self, word: str, top_n: int = 10):
    """Tra về danh sách top_n từ tương đồng (word,
similarity)."""
    if word not in self.model:
        return []
    return self.model.most_similar(word, topn=top_n)

def embed_document(self, document: str) -> np.ndarray:
    """Tạo document embedding bằng cách trung bình các word vector
hợp lệ."""
    tokens = self._basic_tokenize(document)
    vectors = []
    for token in tokens:
        if token in self.model:
            vectors.append(self.model[token])
    if not vectors:
        print("Tài liệu không có token nào nằm trong vocabulary.
Trả về vector 0.")
        return np.zeros(self.vector_size, dtype=float)
    return np.mean(vectors, axis=0)

```

2. Khởi tạo WordEmbedder và kiểm tra nhanh

Dùng mô hình `glove-wiki-gigaword-50` (kích thước 50).

```

embedder = WordEmbedder('glove-wiki-gigaword-50')

[=====] 100.0%
66.0/66.0MB downloaded
Đã tải mô hình 'glove-wiki-gigaword-50' với vector_size = 50

```

Lấy vector của một từ

Thao tác theo yêu cầu: lấy vector của từ king.

```
vector_king = embedder.get_vector('king')
print(f'Chiều dài vector: {len(vector_king)}')
print('10 phân tử'đầu tiên:', np.round(vector_king[:10], 4))
```

Chiều dài vector: 50
10 phân tử'đầu tiên: [0.5045 0.6861 -0.5952 -0.0228 0.6005 -0.135
-0.0881 0.4738 -0.618
-0.3101]

Tính similarity

- king vs queen
- king vs man

```
sim_king_queen = embedder.get_similarity('king', 'queen')
sim_king_man = embedder.get_similarity('king', 'man')

if sim_king_queen is None or sim_king_man is None:
    print("Một trong các từ không nằm trong vocabulary của mô hình.")
else:
    print(f"Similarity(king, queen) = {sim_king_queen:.4f}")
    print(f"Similarity(king, man)    = {sim_king_man:.4f}")
```

Similarity(king, queen) = 0.7839
Similarity(king, man) = 0.5309

Tìm từ tương đồng

Top 10 từ gần nghĩa với computer.

```
most_similar_computer = embedder.get_most_similar('computer',
top_n=10)
for rank, (word, score) in enumerate(most_similar_computer, start=1):
    print(f'{rank:2d}. {word:15s} -> {score:.4f}')
```

1.	computers	-> 0.9165
2.	software	-> 0.8815
3.	technology	-> 0.8526
4.	electronic	-> 0.8126
5.	internet	-> 0.8060
6.	computing	-> 0.8026
7.	devices	-> 0.8016
8.	digital	-> 0.7992
9.	applications	-> 0.7913
10.	pc	-> 0.7883

Tạo document embedding

Theo đề bài: "The queen rules the country."

```
document = 'The queen rules the country.'
doc_vector = embedder.embed_document(document)
print(f'Chiều dài vector tài liệu: {len(doc_vector)}')
print('10 phần tử đầu tiên:', np.round(doc_vector[:10], 4))
print('Norm:', np.linalg.norm(doc_vector))
```

Chiều dài vector tài liệu: 50
10 phần tử đầu tiên: [0.0244 0.378 -0.6382 0.0128 0.0524 0.1195
-0.3165 -0.0878 0.0776
-0.5418]
Norm: 4.237448

3. Bonus: Train Word2Vec từ dữ liệu UD English-EWT

```
from google.colab import drive
import os

# Kê't nôi Google Drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Tải dataset

```
import os
import re
import requests
from zipfile import ZipFile
from io import BytesIO

# Đường dẫn lưu
data_dir = "/content/drive/MyDrive/data_nlp"
os.makedirs(data_dir, exist_ok=True)

# URL gốc của bộ dữ liệu
url =
"https://github.com/UniversalDependencies/UD_English-EWT/archive/refs/heads/master.zip"

print("Đang tải bộ dữ liệu UD English-EWT...")
response = requests.get(url)
response.raise_for_status()

# Giải nén trực tiếp từ bộ nhớ
```

```

with ZipFile(BytesIO(response.content)) as zip_ref:
    for file in zip_ref.namelist():
        if file.endswith("en_ewt-ud-train.conllu"):
            print("Đang trích xuất file en_ewt-ud-train.conllu ...")
            zip_ref.extract(file, data_dir)
            conllu_path = os.path.join(data_dir, file)
            break

# Đọc và trích xuất text
output_path = os.path.join(data_dir, "en_ewt-ud-train.txt")

with open(conllu_path, "r", encoding="utf-8") as fin,
open(output_path, "w", encoding="utf-8") as fout:
    for line in fin:
        if line.startswith("# text = "):
            sentence = line[len("# text = "):].strip()
            fout.write(sentence + "\n")

print(f"Đã tạo file: {output_path}")

Đang tải bộ dữ liệu UD English-EWT...
Đang trích xuất file en_ewt-ud-train.conllu ...
Đã tạo file: /content/drive/MyDrive/data_nlp/en_ewt-ud-train.txt

def load_ud_sentences(ud_path: Path) -> List[List[str]]:
    """Đọc dữ liệu UD ở định dạng CoNLL-U hoặc plain text và trả về`
    list các câu (danh sách token)."""
    if not ud_path.exists():
        raise FileNotFoundError(f"Không tìm thấy tập tin: {ud_path}")

    sentences: List[List[str]] = []
    current_sentence: List[str] = []
    is_conllu: Optional[bool] = None

    with ud_path.open('r', encoding='utf-8') as f:
        for raw_line in f:
            line = raw_line.strip()
            if not line:
                if is_conllu and current_sentence:
                    sentences.append(current_sentence)
                    current_sentence = []
                elif not is_conllu:
                    # Dữ liệu plain text: newline đánh dấu câu mới đã
                    được thêm trực tiếp bên dưới
                    pass
                continue

            if is_conllu is None:
                is_conllu = '\t' in line

```

```

        if is_conllu:
            if line.startswith('#'):
                continue
            parts = line.split('\t')
            if len(parts) > 1:
                token = parts[1].strip()
                if token and token != '_':
                    current_sentence.append(token.lower())
            else:
                tokens = re.findall(r"[\w']+", line.lower())
                if tokens:
                    sentences.append(tokens)

    if is_conllu:
        if current_sentence:
            sentences.append(current_sentence)

    sentences = [sent for sent in sentences if sent]

    if not sentences:
        raise ValueError(
            "Không trích xuất được câu nào từ tập tin. Hãy kiểm tra  

            lại định dạng "  

            "(CoNLL-U hoặc văn bản thuần, mỗi câu một dòng).")
        )

    print(f"Đã đọc {len(sentences)} câu từ {ud_path.name}")
    return sentences

from gensim.models import Word2Vec

def train_custom_word2vec(
    ud_path: Path,
    vector_size: int = 100,
    window: int = 5,
    min_count: int = 2,
    epochs: int = 10,
) -> Word2Vec:
    """Huấn luyện Word2Vec trên dữ liệu UD đã đọc."""
    sentences = load_ud_sentences(ud_path)
    print(f"Tổng số câu sử dụng: {len(sentences)}")

    model = Word2Vec(
        sentences=sentences,
        vector_size=vector_size,
        window=window,
        min_count=min_count,
        workers=4,
    )

```

```

        sg=1,
        epochs=epochs,
    )
    return model

# Đường dẫn mặc định
ud_train_path = Path('/content/drive/MyDrive/data_nlp/en_ewt-ud-
train.txt')

if ud_train_path.exists():
    custom_model = train_custom_word2vec(ud_train_path)
    results_dir = Path('results')
    results_dir.mkdir(exist_ok=True)
    model_path = results_dir / 'word2vec_ewt.model'
    custom_model.save(str(model_path))
    print(f'Đã lưu model tại {model_path}')
    print(custom_model.wv.most_similar('computer', topn=5))
else:
    print('Không tìm thấy file UD. Hãy tải về hoặc điều chỉnh đường
dẫn.')

Đã đọc 12488 câu từ en_ewt-ud-train.txt
Tổng số câu sử dụng: 12488
Đã lưu model tại results/word2vec_ewt.model
[('laptop', 0.9428392648696899), ('recommendation',
0.9401106238365173), ('password', 0.9393160939216614), ('shanna',
0.9383987188339233), ('spray', 0.9351677298545837)]

```

4. Advanced: Word2Vec với Apache Spark

```

import json
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split,
rand
from pyspark.sql.functions import explode
from pyspark.sql import functions as F
from pyspark.ml.feature import Word2Vec as SparkWord2Vec

def run_spark_word2vec(
    json_path: str,
    vector_size: int = 100,
    min_count: int = 5,
    word_for_synonym: str = "technology",
):
    spark =
SparkSession.builder.appName("Lab4SparkWord2Vec").getOrCreate()
    print("Đang đọc dữ liệu JSON...")
    df = spark.read.json(json_path)

```

```

text_col = "text" if "text" in df.columns else df.columns[0]

df_clean = (
    df.select(lower(col(text_col)).alias("text"))
    .withColumn("text", regexp_replace("text", r"^[a-z\s]", " "))
    .withColumn("words", split(col("text"), r"\s+"))
)

tokens_df =
df_clean.select(explode("words").alias("token")).filter(col("token") != "")
token_freq_df =
tokens_df.groupBy("token").agg(F.count("token").alias("freq"))
target_freq_row = token_freq_df.filter(col("token") ==
word_for_synonym).collect()
target_freq = target_freq_row[0]["freq"] if target_freq_row else 0
print(f"Từ '{word_for_synonym}' xuất hiện {target_freq} lần trong
dữ liệu (trước khi áp dụng min_count).")

word2vec = SparkWord2Vec(
    vectorSize=vector_size,
    minCount=min_count,
    inputCol="words",
    outputCol="embedding",
)
model = word2vec.fit(df_clean)

vocab_df = model.getVectors().select("word").cache()
vocab_size = vocab_df.count()
print(f"Kích thước vocabulary sau khi áp dụng
min_count={min_count}: {vocab_size}")

target_in_vocab = vocab_df.filter(col("word") ==
word_for_synonym).limit(1).count() > 0

if target_in_vocab:
    synonyms = model.findSynonyms(word_for_synonym, 5)
    print(f"Top 5 từ giống '{word_for_synonym}':")
    synonyms.show(truncate=False)
else:
    if target_freq == 0:
        print(
            f"Không tìm thấy từ '{word_for_synonym}' trong dữ liệu
nguồn. "
            "Kiểm tra lại cách tiền xử lý hoặc chọn từ khác."
        )
    elif target_freq < min_count:
        print(
            f"Từ '{word_for_synonym}' chỉ xuất hiện {target_freq}
lần, nhỏ hơn min_count={min_count}.\n"

```

```

        "Hãy giảm tham số min_count (ví dụ =1 hoặc =2) và chạy
        lại."
    )
    else:
        print(
            f"Từ '{word_for_synonym}' có {target_freq} lần nhưng
            vẫn không vào vocab. "
            "Hãy kiểm tra dữ liệu hoặc thử min_count nhỏ hơn."
        )

    related_candidates = (
        token_freq_df
        .filter(col("token").like(f"%{word_for_synonym}%"))
        .orderBy(F.desc("freq"))
        .limit(5)
        .collect()
    )
    if related_candidates:
        print("Các token có chứa chuỗi mục tiêu trong dữ liệu và
        tần suất của chúng:")
        for row in related_candidates:
            print(f" {row['token']} -> {row['freq']} lần")
    else:
        print("Không có token nào chứa trực tiếp chuỗi đó trong dữ
        liệu.")

    sample_words = [row.word for row in
    vocab_df.orderBy(rand()).limit(5).collect()]
    if sample_words:
        print("Một vài từ có trong vocabulary đề'thư:",
        sample_words)
        fallback_word = sample_words[0]
        print(f"\nThư'tìm synonym cho từ '{fallback_word}':")
        model.findSynonyms(fallback_word, 5).show(truncate=False)

    spark.stop()

# Ví dụ sử' dụng (đảm bảo' đường dẫn chính xác trước khi chạy)
run_spark_word2vec(
    json_path='/content/drive/MyDrive/data_nlp/c4-train.00000-of-
    01024-30K.json.gz',
    vector_size=100,
    min_count=5,
    word_for_synonym='computer'
)

```

Đang đọc dữ liệu JSON...

Từ 'computer' xuất hiện 1801 lần trong dữ liệu (trước khi áp dụng min_count).

Kích thước vocabulary sau khi áp dụng min_count=5: 50245

Top 5 từ giống 'computer':

word	similarity
computers	0.6919286847114563
desktop	0.6762557625770569
software	0.6182862520217896
programming	0.6033236384391785
microsoft	0.5922807455062866

5. Tổng kết & Đánh giá

- **Thực hiện:** Hoàn thành toàn bộ yêu cầu chính (vector từ, similarity, most similar, document embedding) và mở rộng (huấn luyện Word2Vec riêng, Spark Word2Vec trên tập C4 mẫu).
- **Kết quả chính:** Mô hình pre-trained GloVe cho kết quả ổn định; mô hình tự train trên UD English-EWT học được quan hệ cơ bản; Spark Word2Vec xử lý dữ liệu lớn nhưng nhạy với min_count và tần suất token.
- **Quan sát:** Các từ công nghệ như computer, technology thể hiện tương đồng đúng kỳ vọng khi còn trong vocabulary; document embedding trung bình vector phản ánh tốt ý nghĩa câu mẫu.